

گزارش تمرین کامپیوتری 1 هوش مصنوعی

- یکی از ایده‌ها برای هیوریستیک، استفاده از فاصله منهن است. به این صورت که تابع را به صورت جمع فاصله منهن بازیکن تا جعبه و فاصله منهن جعبه تا مکان نهایی در نظر بگیریم. مشکل استفاده از این هیوریستیک در این مسئله چیست؟
مشکل این است که در این محاسبات بقیه مانع‌ها مانند دیوار‌ها و اینکه دو جعبه پشت هم باشند یا اینکه فاصله مایک از اولین یا نزدیک‌ترین جعبه چقدر هست در نظر گرفته نشده است برای همین در حالت اصلی به احتمال زیاد راه خیلی طولانی‌تر و پیچیده‌تر میشود برای همین تخمین درستی نیست. اگر در تابع هیوریستیک بتوانیم دیوار‌ها و یا پورتال‌ها و یا فاصله مایک از نزدیک‌ترین جعبه را در نظر بگیریم میتوانیم بهینه‌تر تخمین بزنیم.
1. در این سوال مکان جعبه و مکان هدف رو استیت در نظر گرفتیم چون با تغییرات هر کدام حالت جدیدی از بازی تولید میشود که در هر حالت جدید باید چک کنیم که جعبه‌ها در مکان هدف وجود دارد یا نه. اگر نقشه انبار در هر لحظه را در نظر می‌گرفتیم باید حافظه بیشتری رو می‌گرفتیم چون از کل داده‌های نقشه موقعیت بازیکن و جعبه‌ها فقط اهمیت دارد نه دیوار‌ها یا پورتال‌ها که اهمیتی ندارد و ثابت هستند فقط هر دفعه به شکل تکراری اون‌ها رو چک می‌کنیم
2. اکشن‌ها در این بازی حرکت بازیکن در چهار جهت بالا پایین چپ و راست بودند که اگر این حرکت برای ان‌ها مجاز بود با تابع `apply_move()` این حرکت را انجام میدادیم و اگر در اثر ان جعبه‌ای حرکت میکرد تغییراتش بعد اون اعمال میشد.
3. در ابتدا `initial state` برای بازیکن و جعبه‌ها را با خواندن مپ و سیو کردن ان انجام دادیم و از این استیت سرچ را شروع کردیم و در لیست `visited` قرار دادیم سپس با یک حلقه همه استیت‌هایی که داخل `queue` هستند رو چک می‌کنیم. به ترتیب که بیرون می‌کشیم ابتدا چک می‌کنیم که `goal state` هست یا نه (با چک کردن این که هر جعبه در مکان هدف خودش هست یا نه) سپس اگر به هدف نرسیده بودیم همه استیت‌هایی که از استیت فعلی میشه به ان‌ها رسید با انجام دادن `action`‌ها که بالا پایین چپ راست رفتن بازیکن هست استیت‌های جدید رو به `queue` اضافه می‌کنیم بعد از اضافه کردن هر استیت هم حتما باید به استیت فعلی خودمون برگردیم.
4. موردی که این سرچ داشت تعداد زیاد استیت‌ها بود چون وقتی تعداد جعبه‌ها افزایش میابد و مپ بزرگ‌تر میشود حالت‌های بیشتری باید در سرچ بررسی شود و حافظه بیشتری را میگیرد همچنین زمان بیشتری میبرد برای پیدا کردن مسیر

هدف . برای همین اگر بتوان فضا استتیت هارو محدود کرد متلا با سقف هزینه ها هم سرعت افزایش میابد هم حافظه کمتری میگیرد

5. BFS: در این الگوریتم حالات را در عرض گراف بررسی میکنیم و در کد از: BFS استفاده میکنیم این الگوریتم بهینه هست چون جواب در کمترین عمق رو میدهد و همچنین کامل هست و قطعا به جواب میرسد فقط تنها مشکل این الگوریتم حافظه و حرکات زیاد هست در صورتی که فضای حالات بزرگ باشد این الگوریتم در عمق میره برای همین حتما جواب بهینه را نمیده ولی کامله: DSF . در مقایسه با الگوریتم قبلی فضای حالات بیشتری را طی میکند و حافظه بیشتری را میگیرد . در کل جواب نهایی رو میدهد ولی نسبت به بقیه الگوریتم ها بهینه نیست
- این الگوریتم از قبلی بهینه ترهست چون هر بار با عمق مشخص همه: IDS حالات رو بررسی میکند و چون فقط حالات های همون عمق رو ذخیره میکند حافظه کمتری را میگیرد BFS نسبت به
- نسبت به بقیه الگوریتم ها بهینه تر هست چون آگاهانه هست و با بررسی: A* حالات یک استتیت را گسترش میدهد و در مسیر هدف استتیت هارو بررسی میکند و بستگی به تابع هیوریستیک که انتخاب میکنیم دقت ان تعبیر میکند ولی در نهایت باشد ADMISSIBLE بهینه هست و اگر در تابع هیوریستیک حواسمان باید (جواب بهینه را به ما میدهد (یعنی تخمین درست
- الگوریتم خوبی نیست چون مسیر بهینه رو برنمیگرداند و چون اولین عمق رو: A ادامه میدهد اگر فضای حالت بزرگ باشد امکان دارد به جواب نرسد و یا حتی در یک لوپ گیر کند . ولی برای مسایلی با فضای محدود و عمق کم خوب هست چون پیاده سازی ساده ای دارد و حافظه کمتری رو میگیرد
- این الگوریتم مزیت های دو الگوریتم قبلی را دارد یعنی از بی اف اس بهینه: B بودن و کامل بودن و از دی اف اس سادگی و حافظه کم را دارد . این الگوریتم با محدود کردن عمق در هر سرچ هم باعث میشود که حافظه کمتر درگیر شود چون فقط داده های یک عمق را نگه داری میکند هم بهینه هست چون در عمق کمتر اگر جواب وجود داشته باشد ان را برمیگرداند

6.

```
▶ solve(1, "BFS") # Solve map 1 using BFS
[123] ✓ 0.0s
... BFS took 0.01 seconds on map 1 and visited 45 states.
7 moves were used: UDDURLL
```

```
▶ solve(1, "DFS") # Solve map 1 using BFS
[124] ✓ 0.0s
... DFS took 0.01 seconds on map 1 and visited 12 states.
8 moves were used: RLLRDUUD
```

```
▶ solve(1, "IDS") # Solve map 1 using BFS
[125] ✓ 0.0s
... IDS took 0.03 seconds on map 1 and visited 104 states.
7 moves were used: UDDULRR
```

طبق پیش بینی DFS حالت بهینه رو برنگرداند چون اول در عمق جست و جو میکند ولی دو الگوریتم دیگر حالت های بهینه رو برگرداندن که فکر کنم توی این حالت چون پیچیدگی کمی داره حافظه BFS کمتر شده از IDS .7 من در هیوریستیک جدید نقطه شروع رو از نزدیک ترین جعبه زدم . که بازیکن از جابجایی نزدیک ترین جعبه خود شروع کند که **admissible** هستن چون کمترین مقدار هزینه رو تخمین میزنن و بیش از حد نمیتونن تخمین بزنن چون پورتال ها و دیوار ها و کنار هم بودن دو جعبه رو در نظر نگرفته ایم . شرط **consistent** رو هم رعایت میکند چون برای هیوریستیک داریم فاصله هارو در نظر میگیریم و با جابجایی بین دو استیت فاصله بین ان دو هم در نظر گرفته میشه

```

solve(5, "A*") # Solve map 1 using BFS
[149] ✓ 0.2s
... A* took 0.27 seconds on map 5 and visited 535 states.
    15 moves were used: LULDDRDLUUURUL

```

.8

```

solve(5, "A*") # Solve map 1 using BFS
[133] ✓ 0.1s
... A* took 0.18 seconds on map 5 and visited 401 states.
    15 moves were used: LULDDRDLUUURUL

```

با هیوریستیک اولیه

با هیوریستیک نزدیک ترین فاصله بازیکن

.9

WEIGHT = 2

MAP1:

```

solve(1, "BFS") # Solve map 1 using BFS
[152] ✓ 0.0s
... BFS took 0.01 seconds on map 1 and visited 45 states.
    7 moves were used: UDDURLL

```

```

solve(1, "DFS") # Solve map 1 using BFS
53] ✓ 0.0s
... DFS took 0.0 seconds on map 1 and visited 12 states.
    8 moves were used: RLLRDUUD

```

```
> solve(1, "IDS") # Solve map 1 using BFS  
(ctrl+Alt+Enter)  
IDS took 0.03 seconds on map 1 and visited 104 states.  
7 moves were used: UDDULRR
```

```
> solve(1, "A*") # Solve map 1 using BFS  
5] ✓ 0.0s  
A* took 0.02 seconds on map 1 and visited 44 states.  
7 moves were used: UDDULRR
```

```
> solve(1, "A*") # Solve map 1 using BFS  
174] ✓ 0.0s  
A* took 0.0 seconds on map 1 and visited 41 states.  
7 moves were used: UDDULRR
```

MAP2:

```
> solve(2, "BFS") # Solve map 1 using BFS  
(Alt+Enter)  
BFS took 0.01 seconds on map 2 and visited 22 states.  
6 moves were used: LUDDUR
```

```
• solve(2, "DFS") # Solve map 1 using BFS  
✓ 0.0s  
DFS took 0.0 seconds on map 2 and visited 11 states.  
7 moves were used: LLRDUUD
```

```
solve(2, "IDS") # Solve map 1 using BFS
```

✓ 0.0s

IDS took 0.01 seconds on map 2 and visited 39 states.
6 moves were used: LUDDUL

```
solve(2, "A*") # Solve map 1 using BFS
```

✓ 0.0s

A* took 0.0 seconds on map 2 and visited 18 states.
6 moves were used: LUDDUL

```
solve(2, "A*") # Solve map 1 using BFS
```

✓ 0.0s

A* took 0.01 seconds on map 2 and visited 16 states.
6 moves were used: LUDDUL

MAP 3:

```
solve(3, "BFS") # Solve map 1 using BFS
```

✓ 0.0s

BFS took 0.03 seconds on map 3 and visited 123 states.
14 moves were used: ULUURDDDULDUIIU

```
solve(3, "DFS") # Solve map 1 using BFS
```

✓ 0.0s

DFS took 0.01 seconds on map 3 and visited 42 states.
14 moves were used: ULDDUIIUURDDDDU

```
solve(3, "IDS") # Solve map 1 using BFS
```

✓ 0.1s

IDS took 0.1 seconds on map 3 and visited 479 states.
13 moves were used: ULDDUIIUURDDDD

```
solve(3, "A*") # Solve map 1 using BFS
```

```
✓ 0.0s
```

```
A* took 0.01 seconds on map 3 and visited 64 states.  
14 moves were used: ULUURDDDDUULDD
```

```
solve(3, "A*") # Solve map 1 using BFS
```

```
✓ 0.0s
```

```
A* took 0.01 seconds on map 3 and visited 35 states.  
14 moves were used: ULUURDDDDUULDD
```

MAP 4: